

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по практической работе №1
по дисциплине
«Введение в аппаратное обеспечение систем искусственного интеллекта»
«Вариант 14»

Выполнил студент гр. ИВТб-1303-06-00 _____ /Гортоломей И.К./
Проверил доцент кафедры ЭВМ _____ /Коржавина А.С./

Киров
2026

Цель работы:

Научиться управлять цифровыми/аналоговыми портами, работать с базовой периферией без блокирующих алгоритмов.

Решение заданий

1 Бегущий огонек

Начальное состояние: выкл

Движение: От крайних к середине

Порядок: 09-18-27-36-45-36-27-18-09

Код прошивки:

```
#define FIRST_LED_PIN 2
#define LAST_LED_PIN 11

void setup() {
    for (int pin = FIRST_LED_PIN; pin <= LAST_LED_PIN; ++pin)
        pinMode(pin, OUTPUT);
}

void loop() {
    unsigned int ms = millis();
    int i = (ms / 120) % 10;
    if (i > 4) i = 9 - i;
    digitalWrite(FIRST_LED_PIN + i, HIGH);
    digitalWrite(FIRST_LED_PIN + (9 - i), HIGH);
    delay(10);
    digitalWrite(FIRST_LED_PIN + i, LOW);
    digitalWrite(FIRST_LED_PIN + (9 - i), LOW);
}
```

i - это число от 0 до 9, которое используется для прямого и обратного "хода" светодиода левой половины и это отзеркаливается на правую половину.

2 Пульсар

Начальное состояние: выкл

Транзистор: Полевой

Смена состояний: Увеличение до середины/сброс/Увеличение max

Код прошивки:

```
#define CONTROL_PIN 3

int brightness = 0;
void setup() {
    pinMode(CONTROL_PIN, OUTPUT);
}

bool to_half = true;
void loop() {
    analogWrite(CONTROL_PIN, brightness);
    delay(10);
    brightness++;
    if (brightness > 127 && to_half) {
        brightness = 0;
        to_half = false;
    }
    if (brightness > 255) {
        brightness = 0;
        to_half = true;
    }
}
```

to_half является флагом для прохода до 127, потом сбрасывается и счётчик идёт до 255.

3 Ночной светильник

Начальное состояние: вкл

Входной сигнал: фоторезистор

Делитель напряжения: 10 кОм

Смена состояний: Уменьшение в зависимости
от значения на датчике

Код прошивки:

```
#define LED_PIN 3
#define RESISTOR_PIN A0

void setup() {
    pinMode(LED_PIN, OUTPUT);
    pinMode(RESISTOR_PIN, INPUT);
}

// >1000 - макс.
// 890-910 - ср.
// 160-350 - мин.
void loop() {
    int x = analogRead(RESISTOR_PIN);
    x -= 200;
    if (x < 0) x=0;
    if (x > 800) x=800;
    float p = (800 - x) / 800.0;
    unsigned char l = (255 * p);
    analogWrite(LED_PIN, l);
}
```

```
    delay(100);  
}
```

Получаемый с пина A0 сигнал от фоторезистора в аудитории в среднем имеет значение 890-910, при полном закрытии он выдаёт 160-350, а при свете фонарика он максимум выдаст 1000-1010. Для максимальной и минимальной яркости светодиода используется вычитание и проверки значения с датчика, а также инвертируется для того чтобы яркость светодиода уменьшалась по мере увеличения значения с датчика.

4 Азбука Морзе

Кнопка: Срабатывание по нажатию

Файл melody.h:

```
#define NOTE_B0  31  
#define NOTE_C1  33  
#define NOTE_CS1 35  
#define NOTE_D1  37  
#define NOTE_DS1 39  
#define NOTE_E1  41  
#define NOTE_F1  44  
#define NOTE_FS1 46  
#define NOTE_G1  49  
#define NOTE_GS1 52  
#define NOTE_A1  55  
#define NOTE_AS1 58  
#define NOTE_B1  62  
#define NOTE_C2  65  
#define NOTE_CS2 69  
#define NOTE_D2  73
```

```
#define NOTE_DS2 78
#define NOTE_E2 82
#define NOTE_F2 87
#define NOTE_FS2 93
#define NOTE_G2 98
#define NOTE_GS2 104
#define NOTE_A2 110
#define NOTE_AS2 117
#define NOTE_B2 123
#define NOTE_C3 131
#define NOTE_CS3 139
#define NOTE_D3 147
#define NOTE_DS3 156
#define NOTE_E3 165
#define NOTE_F3 175
#define NOTE_FS3 185
#define NOTE_G3 196
#define NOTE_GS3 208
#define NOTE_A3 220
#define NOTE_AS3 233
#define NOTE_B3 247
#define NOTE_C4 262
#define NOTE_CS4 277
#define NOTE_D4 294
#define NOTE_DS4 311
#define NOTE_E4 330
#define NOTE_F4 349 // ->
#define NOTE_FS4 370
#define NOTE_G4 392
#define NOTE_GS4 415
```

```
#define NOTE_A4 440
#define NOTE_AS4 466
#define NOTE_B4 494
#define NOTE_C5 523
#define NOTE_CS5 554
#define NOTE_D5 587
#define NOTE_DS5 622
#define NOTE_E5 659
#define NOTE_F5 698
#define NOTE_FS5 740 // ->
#define NOTE_G5 784
#define NOTE_GS5 831
#define NOTE_A5 880
#define NOTE_AS5 932
#define NOTE_B5 988
#define NOTE_C6 1047
#define NOTE_CS6 1109
#define NOTE_D6 1175
#define NOTE_DS6 1245
#define NOTE_E6 1319 // !!!
#define NOTE_F6 1397
#define NOTE_FS6 1480
#define NOTE_G6 1568
#define NOTE_GS6 1661
#define NOTE_A6 1760
#define NOTE_AS6 1865
#define NOTE_B6 1976
#define NOTE_C7 2093 // <-
#define NOTE_CS7 2217
#define NOTE_D7 2349
```

```

#define NOTE_DS7 2489
#define NOTE_E7 2637
#define NOTE_F7 2794
#define NOTE_FS7 2960
#define NOTE_G7 3136
#define NOTE_GS7 3322
#define NOTE_A7 3520
#define NOTE_AS7 3729
#define NOTE_B7 3951
#define NOTE_C8 4186
#define NOTE_CS8 4435
#define NOTE_D8 4699
#define NOTE_DS8 4978

```

```

typedef struct {
    unsigned short note;
    unsigned char temp;
} Block;

```

```

// noname

```

```

Block melody_1[] = {
    {NOTE_AS5, 8}, {NOTE_AS5, 8}, {NOTE_AS5, 8},
    {NOTE_FS5, 4}, {NOTE_CS6, 4},
    {NOTE_AS5, 8}, {NOTE_FS5, 8}, {NOTE_CS6, 4},
    {NOTE_AS5, 2},
    {NOTE_F6, 8}, {NOTE_F6, 8}, {NOTE_F6, 8},
    {NOTE_FS6, 4}, {NOTE_CS6, 4},
    {NOTE_AS5, 8}, {NOTE_FS5, 8}, {NOTE_CS6, 4},
    {NOTE_AS5, 2}
};

```



```

const int melody_1_size = sizeof(melody_1) / sizeof(melody_1[0]);

// mario
Block melody_2[] = {
    {NOTE_E7, 12}, {NOTE_E7, 12}, {0, 12}, {NOTE_E7, 12},
    {0, 12}, {NOTE_C7, 12}, {NOTE_E7, 12}, {0, 12},
    {NOTE_G7, 12}, {0, 12}, {0, 12}, {0, 12},
    {NOTE_G6, 12}, {0, 12}, {0, 12}, {0, 12},

    {NOTE_C7, 12}, {0, 12}, {0, 12}, {NOTE_G6, 12},
    {0, 12}, {0, 12}, {NOTE_E6, 12}, {0, 12},
    {0, 12}, {NOTE_A6, 12}, {0, 12}, {NOTE_B6, 12},
    {0, 12}, {NOTE_AS6, 12}, {NOTE_A6, 12}, {0, 12},

    {NOTE_G6, 9}, {NOTE_E7, 9}, {NOTE_G7, 9},
    {NOTE_A7, 12}, {0, 12}, {NOTE_F7, 12}, {NOTE_G7, 12},
    {0, 12}, {NOTE_E7, 12}, {0, 12}, {NOTE_C7, 12},
    {NOTE_D7, 12}, {NOTE_B6, 12}, {0, 12}, {0, 12},

    {NOTE_C7, 12}, {0, 12}, {0, 12}, {NOTE_G6, 12},
    {0, 12}, {0, 12}, {NOTE_E6, 12}, {0, 12},
    {0, 12}, {NOTE_A6, 12}, {0, 12}, {NOTE_B6, 12},
    {0, 12}, {NOTE_AS6, 12}, {NOTE_A6, 12}, {0, 12},

    {NOTE_G6, 9}, {NOTE_E7, 9}, {NOTE_G7, 9},
    {NOTE_A7, 12}, {0, 12}, {NOTE_F7, 12}, {NOTE_G7, 12},
    {0, 12}, {NOTE_E7, 12}, {0, 12}, {NOTE_C7, 12},
    {NOTE_D7, 12}, {NOTE_B6, 12}, {0, 12}, {0, 12}
};

const int melody_2_size = sizeof(melody_2) / sizeof(melody_2[0]);

```

Код прошивки:

```
#include "melody.h"

#define BUZZER_PIN 9

bool running = false;
bool first_selected = true;
unsigned int thisNote = 0;

void f1() {
    first_selected = !first_selected;
    running = false;
    thisNote = 0;
}

void f2() {
    running = !running;
}

void setup() {
    attachInterrupt(INT0, f1, RISING);
    attachInterrupt(INT1, f2, RISING);
    for (int i = 0; i < 3; i++)
        pinMode(4+i, OUTPUT);
}

Block *melody;
unsigned int melody_size;

void loop() {
```

```

digitalWrite(4, first_selected);
digitalWrite(5, !first_selected);

if (!running) return;

if (first_selected) {
    melody = melody_1;
    melody_size = melody_1_size;
} else {
    melody = melody_2;
    melody_size = melody_2_size;
}

if (thisNote >= melody_size) {
    thisNote = 0;
    running = false;
    return;
};

int noteDuration = 1000 / melody[thisNote].temp;
tone(BUZZER_PIN, melody[thisNote].note, noteDuration);
digitalWrite(6, melody[thisNote].note > 0);
delay(noteDuration);
digitalWrite(6, 0);
delay(noteDuration * 0.3);
noTone(BUZZER_PIN);

thisNote++;
}

```

Первая кнопка переключает мелодию, также останавливает текущую мелодию

и сбрасывает указатель. В "melody.h" хранятся мелодии в виде нот, то есть, в массиве где записана частота ноты и её задержка. Флаги "first_selected" и "running" отвечают за выбор и воспроизведение мелодии соответственно. В зависимости от выбранной мелодии загорается первый или второй светодиод, а также третий светодиод мигает когда звучит не нулевая частота.

5 Терменвокс

При падении освещенности звук: Уменьшается

Звуковой сигнал: 10 раз в сек.

Частоты нот: Малая октава

Код прошивки:

```
#define BUZZER_PIN 3
```

```
#define LDR_PIN A0
```

```
void setup() {  
  pinMode(BUZZER_PIN, OUTPUT);  
}
```

```
void loop() {  
  int val = analogRead(LDR_PIN);  
  int frequency = map(val, 400, 500, 130, 247);  
  tone(BUZZER_PIN, frequency, 20);  
  delay(100);  
}
```

"frequency" хранит частоту, которую мы получаем с термистора путём преобразования его диапазона в диапазон частот Малой октавы. На термисторе выбран диапазон 400 - 500 т.к. в аудитории он остывает и стремится к 500, а при нагреве пальцами стремится к 400.

6 Мерзкое пианино

Подключение кнопок: Без резистора

Входной сигнал: массив

Частоты нот: Малая октава

Код прошивки:

```
#define BUZZER_PIN 9
#define BUTTON_START 2
#define BUTTON_COUNT 5

void setup() {
    pinMode(BUZZER_PIN, OUTPUT);
    for (int i = 0; i < BUTTON_COUNT; i++)
        pinMode(BUTTON_START + i, INPUT_PULLUP);
}

volatile bool b[5];
volatile unsigned char index = 0;
const unsigned short m[] = {
    130, 147, 164, 174, 196,
};

void loop() {
    if (index >= BUTTON_COUNT) index = 0;
    bool btn = digitalRead(BUTTON_START + index);
    if (!btn && b[index])
        tone(BUZZER_PIN, m[index], 500);
    b[index] = btn;
    index++;
}
```

"index" - как указатель в цикле, проходит по значениям от 0 до 4. в "b" хранятся предыдущие состояния кнопок нажата/не нажата, а в "m" хранятся частоты. в "btn" записывается текщее состояние текущей кнопки и сравнивается с предыдущим её состоянием. Для реализации без резистора была добавлена настройка пинов на встроенный резистор, чтобы небыло помех.

Выводы

Выполняя данную работу я научился работать с пинами цифровыми и аналоговыми. Также научился работать с периферией не блокируя основной цикл, в основном используя прерывания и таймеры. Научился управлять большим током через транзистор и считывать данные с датчиков.